

Identificación musical estilo Shazam: Huellas acústicas y espectrogramas

Introducción

En los últimos años se han popularizado aplicaciones capaces de identificar una canción a partir de un breve fragmento de audio, siendo **Shazam** el ejemplo más famoso. Estas aplicaciones utilizan el concepto de **huella digital acústica**: un identificador único extraído del contenido sonoro del archivo de audio es.wikipedia.org. En esencia, convierten el audio en una “firma” o patrón característico que luego puede compararse contra una gran base de datos de huellas para encontrar coincidencias. Este enfoque permite reconocer canciones sin necesidad de metadatos, incluso con grabaciones de pocos segundos y con ruido de fondo es.wikipedia.org.

El algoritmo original de Shazam (patentado por Avery Li-Chun Wang en 2003) está basado en análisis espectral. A grandes rasgos, **transforma la señal de audio en un espectrograma**, extrae los picos más prominentes de frecuencia en el tiempo, genera con ellos una huella digital compacta (normalmente mediante un *hash* numérico) y luego busca eficientemente esa huella en la base de datos de canciones conocidas [toptal.com](https://www.toptal.com). A continuación, se detalla cada etapa del proceso, los datos mínimos necesarios, cómo se construye la huella y cómo se realiza la comparación eficiente, junto con recomendaciones prácticas de implementación y fuentes de datos abiertas para construir un prototipo.

Análisis espectral de la señal de audio (dominio de tiempo a dominio de frecuencia)

El **primer paso** para obtener la huella digital acústica es convertir el audio del dominio temporal al **dominio frecuencial**. Una canción digital es básicamente una secuencia de muestras en el tiempo (por ejemplo, 44,100 muestras por segundo en calidad CD). Mediante la **Transformada Rápida de Fourier (FFT)** aplicada a fragmentos sucesivos de la señal, se construye un **espectrograma**: una representación 2D de la intensidad del sonido en función de la frecuencia y el tiempo willdrevo.com. En el espectrograma, el eje horizontal representa el tiempo, el eje vertical la frecuencia, y la intensidad o amplitud se indica con colores o niveles de brillo (por ejemplo, colores cálidos para mayor amplitud) willdrevo.com.

Figura 1: Espectrograma de unos segundos iniciales de una canción (“Blurred Lines” de Robin Thicke). En esta visualización tiempo-frecuencia, las regiones de alta intensidad (colores más brillantes) indican qué frecuencias dominan en cada instante willdrevo.com. Se aprecia que la información de audio se distribuye a lo largo de distintas frecuencias que varían con el tiempo (por ejemplo, notas musicales, ritmo, timbres de instrumentos). El espectrograma proporciona así la “huella” visual de la canción en términos de contenido frecuencial.

Para construir el espectrograma digitalmente, es común usar la **Transformada de Fourier de tiempo corto (STFT)**, dividiendo la señal en ventanas temporales (por ejemplo, segmentos de 0.1 s) posiblemente solapadas, aplicando la FFT a cada segmento. Esto produce una matriz tiempo-frecuencia cuyos valores son las magnitudes espectrales. Es útil trabajar con la escala logarítmica de magnitud (decibelios) para resaltar las componentes significativas. Además, dado que el oído humano percibe frecuencia de forma logarítmica, a veces se usa una escala de frecuencia mel o logarítmica; sin embargo, el algoritmo de Shazam original trabaja en frecuencia lineal pero enfocándose en picos locales. **Datos mínimos necesarios hasta este punto:** la tasa de muestreo del audio (ej. 44.1 kHz) y los valores del espectrograma (frecuencia vs tiempo vs amplitud).

Extracción de características: picos de frecuencia dominantes

El **desafío clave** tras obtener el espectrograma es reducir la enorme cantidad de datos (miles de valores por segundo) a unos pocos *descriptores robustos* que caractericen de forma única la canción. Shazam logra esto identificando los **picos de mayor intensidad** en el espectrograma, asumiendo que las frecuencias más fuertes en cada momento constituyen una firma distintiva y menos sensible al ruido toptal.comes.wikipedia.org. En otras palabras, se filtra el “océano” de frecuencias capturadas para quedarse solo con aquellas componentes prominentes.

¿Cómo se extraen los picos? Una estrategia básica es dividir el rango de frecuencias en varias bandas y, para cada ventana temporal, localizar la frecuencia con máxima amplitud en cada banda toptal.com toptal.com. Por ejemplo, una implementación sencilla podría usar bandas como: 40–80 Hz, 80–120 Hz, 120–180 Hz, 180–300 Hz, 300–... Hz (y así sucesivamente hasta cubrir el espectro relevante) toptal.com. En cada intervalo se identifica el pico (la frecuencia con mayor magnitud). De este modo, cada segmento de tiempo queda resumido por un conjunto de frecuencias pico. Tomadas en conjunto, estas frecuencias pico en distintos tiempos actúan como “puntos estelares” en una constelación que caracteriza la canción willdrevo.com.

Otra forma, más aproximada al método original de Shazam, es considerar el espectrograma como imagen y detectar *máximos locales 2D*: puntos (tiempo, frecuencia) cuya magnitud es mayor que la de sus vecinos en un entorno cercano. Esto se puede lograr con técnicas de procesamiento de imágenes (filtro pasa-alto para resaltar altas amplitudes, seguido de detección de máximos locales) willdrevo.com. El resultado es un conjunto disperso de puntos tiempo-frecuencia que sobresalen claramente del ruido y del resto de la señal. **Estos puntos de alta energía son la base de la huella digital.** Al ignorar amplitudes exactas y solo guardar la presencia de picos, el sistema se vuelve más robusto a variaciones de volumen, ruido de fondo o distorsiones (que generalmente no afectan la localización de los picos más fuertes) willdrevo.com. Tras esta etapa, la canción queda representada por pares (tiempo, frecuencia) de sus picos principales. En promedio, una canción típica puede tener decenas de miles de tales picos willdrevo.com willdrevo.com, aunque para huellas más compactas se puede limitar el número de picos por segundo.

Generación de la huella digital acústica (fingerprint)

Obtenidos los picos espectrales, el siguiente paso es combinarlos en una representación compacta que pueda almacenarse y compararse fácilmente: la **huella digital**. Shazam emplea un ingenioso método de *hashing* que combina las frecuencias y diferencias de tiempo entre picos para generar identificadores únicos. En concreto, el algoritmo original forma **pares de picos**: toma un pico como ancla y otro pico cercano en el futuro (dentro de una ventana de tiempo fija, por ejemplo 1–5 segundos) y construye un tupla (f_1 , f_2 , Δt) donde f_1 y f_2 son las frecuencias de ambos picos y Δt es la diferencia en segundos entre sus tiempos willdrevo.com. Esta tupla se codifica luego como un número hash (por ejemplo, combinando bits de f_1 , f_2 y Δt) que sirve de clave en la base de datos willdrevo.com willdrevo.com. El razonamiento es que una combinación de dos picos más su separación temporal es mucho más distintiva que un solo pico aislado, reduciendo la probabilidad de colisión (que dos canciones compartan la misma huella) willdrevo.com. La literatura describe esta colección de picos emparejados como una “constelación de estrellas” única para cada canción willdrevo.com.

Ejemplo: si en un fragmento de la canción se detectan picos a 1800 Hz en $t=12.0$ s y 2400 Hz en $t=12.3$ s, se podría formar un hash con ($f_1=1800$, $f_2=2400$, $\Delta t=0.3$ s). Con una cuantificación adecuada (por ejemplo, redondeando frecuencias a múltiplos de 10 Hz y tiempo a 0.1 s para tolerancia), este hash podría representarse como un entero de 32 o 64 bits. En implementaciones prácticas se suele aplicar un pequeño factor de fuzz o tolerancia en las frecuencias antes de hash toptal.com, para absorber pequeñas desviaciones debidas al ruido o al muestreo.

Hay múltiples variantes para generar el fingerprint. Una implementación simplificada presentada por desarrolladores consiste en tomar los picos máximos de varias bandas en un instante y concatenar esas frecuencias como una sola clave hash toptal.com. Por ejemplo, combinando 4 ó 5 picos de bandas distintas en un segmento de tiempo se obtiene una firma para ese momento toptal.com toptal.com. La figura abajo ilustra un ejemplo de huella simplificada basada en 5 picos por segmento:

Figura 2: Ejemplo de **hash generado** a partir de picos espectrales en diferentes bandas de frecuencia toptal.com toptal.com. Aquí se usan 5 frecuencias pico por ventana temporal (una por cada banda definida). Estas cinco frecuencias –tras aplicar una pequeña cuantización (“FUZ_FACTOR”)– se empaquetan en un solo número entero que actúa como la huella de ese segmento. Este hash, junto con el instante temporal y el ID de la canción, se almacena en la base de datos como se ve en la tabla de ejemplo toptal.com.

Cada canción, entonces, al procesarse, genera *múltiples huellas* (muchos hashes distintos a lo largo de su duración). **El conjunto de todos esos hashes es la huella digital completa de la canción.** Por ejemplo, en la tabla ilustrativa de la Figura 2, “Song A” tiene hashes como 30–51–99–121–195 en distintos tiempos (53.52 s, 54.52 s, etc.), “Song B” tiene otro patrón, etc. toptal.com. Procesando así una biblioteca entera de canciones, se construye una base de datos con entradas del tipo: clave hash -> [(canción, tiempo), (canción, tiempo), ...] toptal.com toptal.com.

Eficiencia de almacenamiento: Nótese que almacenar la huella es mucho más económico que almacenar todo el audio. Cada hash puede ser reducido a, por ejemplo, 32–64 bits. Incluso cientos de miles de hashes ocupan pocos megabytes. En implementaciones reales, se han utilizado hashes de 64 bits (ej. derivados de SHA-1 recortados) combinados con offsets de tiempo, logrando on the order of 5 million huellas por GB willdrevo.com willdrevo.com. Esto hace viable manejar bases de datos de millones de canciones con huellas relativamente pequeñas.

Búsqueda eficiente en la base de datos de huellas

Una vez que las canciones conocidas están indexadas en la base de datos mediante sus huellas (hashes), la identificación de un fragmento desconocido se reduce a un **problema de búsqueda de claves**. El proceso típico es:

1. **Grabación de la muestra:** El usuario graba unos segundos de la canción desconocida (la app captura audio de 5–10 s mediante el micrófono del teléfono). Conviene muestrear en mono, 44.1 KHz, para mantener consistencia con la base de datos toptal.com toptal.com.
2. **Generación de huellas de la muestra:** El fragmento grabado se procesa con el **mismo pipeline** descrito: calculamos su espectrograma, extraemos los picos dominantes y generamos hashes/fingerprints para ese corto intervalo toptal.com. Esto produce una lista de huellas candidate (por ejemplo, algunas centenas de hashes para 5 segundos de audio).
3. **Búsqueda de coincidencias:** Por cada hash obtenido de la muestra, se consulta la base de datos para ver si ese hash existe y a qué canciones corresponde toptal.com. Dado que los hashes fueron indexados, esta consulta es extremadamente rápida. En una base de datos bien diseñada, los hashes pueden ser clave primaria o tener un índice dedicado willdrevo.com willdrevo.com, de modo que buscar un hash es operación $O(1)$ (búsqueda por llave en tabla hash) o $O(\log N)$ (búsqueda en índice B-tree), incluso con millones de entradas. **Así se evitan comparaciones exhaustivas de señales**, que serían inviables a gran escala.
4. **Votación y alineamiento temporal:** Cada hash de la muestra retorna potencialmente varios candidatos (canción X en tiempo t_1 , canción Y en t_2 , etc.), puesto que no es imposible que diferentes canciones compartan algún patrón corto por casualidad o por similitudes musicales toptal.com. Sin embargo, la probabilidad de que muchas huellas coincidan *y además con la misma relación temporal* es virtualmente única para la canción correcta. Por ello, el siguiente paso es una agregación: se agrupan las coincidencias por *song ID* y se analiza la distribución de los tiempos. En particular, se busca el **offset consistente**: si la muestra grabada proviene de la canción X, entonces los tiempos relativos entre huellas en la muestra deben corresponder a desplazamientos constantes con respecto a los tiempos donde aparecen esas mismas huellas en la canción X toptal.com. Por ejemplo, si el hash **H1** del fragmento coincide con la canción A en el segundo 50 de A, y otro hash **H2** de la muestra coincide con canción A en el segundo 53 (y en la muestra H1 y H2

están separados por 3 segundos), eso sugiere que en la canción A, esos hashes ocurren con la misma separación temporal que en la muestra toptal.com. Al acumular muchos de estos pareos, la canción correcta se delata porque tendrá **docenas de coincidencias cuyas diferencias de tiempo concuerdan todas** con un mismo desplazamiento. En cambio, otras canciones pueden compartir uno o dos hashes, pero no la secuencia temporal completa.

En la práctica, Shazam determina la canción identificando cuál produce el mayor número de coincidencias alineadas en el mismo offset de tiempo toptal.com. Esta es típicamente la única que “sobrevive” tras la votación. Finalmente, el sistema devuelve el resultado (título, artista) al usuario.

Nota: Gracias a este esquema, Shazam puede reconocer canciones en ~5 segundos con alta precisión. Si la muestra es ruidosa o muy corta, podría haber casos ambiguos, pero por los requisitos del algoritmo se espera robustez ante ruido, compresión, efectos, etc. es.wikipedia.org, siempre que los picos clave no se vean completamente enmascarados.

Datasets públicos para construir la base de datos de canciones

Para desarrollar un prototipo tipo Shazam, se necesita una **base de datos de referencia** de canciones con la cual comparar las muestras. Idealmente, esta base debe contener **los archivos de audio** (o al menos sus huellas previamente calculadas) de un conjunto amplio de canciones. Afortunadamente, existen varios *datasets* públicos y fuentes de audio que pueden emplearse con fines académicos o demostrativos:

- **GTZAN Genre Collection:** Un conjunto clásico de 1000 fragmentos de audio (cada uno de 30 segundos) repartidos en 10 géneros musicales diferentes, con 100 pistas por género tensorflow.org. Aunque fue diseñado para clasificación de género, se puede utilizar para identificación: cada pista puede considerarse una “canción” única en la base de datos. Ventaja: tamaño manejable (~1 GB) y ampliamente utilizado en investigación. Desventaja: son fragmentos de canciones (no temas completos) y algunos contienen ruido o están repetidos, pero sirve para pruebas iniciales.
- **FMA (Free Music Archive) Dataset:** Una extensa colección de música de dominio público o con licencia Creative Commons. FMA ofrece hasta 106,574 pistas de audio completas, sumando ~343 días de música (~917 GB) archive.ics.uci.edu. Está organizado en subsets: *FMA Small* (8 géneros, 8000 clips), *FMA Medium* (~25,000 clips) y *FMA Large/Full* (106k clips). Todas las pistas tienen licencias libres (CC-BY, CC0, etc.), lo que facilita su uso archive.ics.uci.edu. Para un prototipo, FMA Small o Medium son buenas opciones: por ejemplo, *FMA Small* contiene 8 géneros con 1000 pistas cada uno (30s por pista). Usar FMA permite probar el algoritmo con música real de distintos estilos sin problemas legales.
- **AudioSet (Google):** Un corpus de 2 millones de clips de audio de 10s extraídos de YouTube con anotaciones de contenido innovatiana.com. Incluye categorías de

música y podría utilizarse para obtener fragmentos de canciones populares. Sin embargo, los clips permanecen en YouTube y su calidad varía; es más útil para tareas de clasificación de sonidos que para identificación exacta de canciones enteras.

- **Jamendo/Open Music APIs:** Plataformas como Jamendo ofrecen API para descargar canciones completas con licencias libres. Esto permite construir una base de datos personalizada seleccionando artistas o géneros específicos. Por ejemplo, podrías recolectar 500 canciones rock y 500 pop libres de Jamendo para montar una demo de identificación entre esos géneros.
- **Grabaciones propias o de dominio público:** También es viable usar canciones clásicas de dominio público (p. ej., interpretaciones de música clásica, himnos, etc.) o incluso **samples cortos** de canciones comerciales a modo de referencia educativa. Si la universidad proporciona ciertos audios (como en este caso, 10 archivos de dos géneros predefinidos), se pueden incluir en la base de datos. Sin embargo, para escalar a más canciones es preferible usar datasets abiertos por cuestiones de licencia.

Ejemplo ilustrativo: El proyecto *MIR Audio Identification* de Queen Mary University (2021) implementó un sistema tipo Shazam usando Python/librosa y empleó GTZAN como dataset de referencia para fingerprint github.com. Esto demuestra que datasets estándar de investigación sirven para construir prototipos funcionales de identificación musical.

Pipeline técnico en Python para un prototipo

A continuación se describe un **pipeline paso a paso** para implementar un prototipo de identificador musical en Python. Este pipeline hace uso de bibliotecas comunes de procesamiento de audio (p. ej., *NumPy*, *SciPy*, *librosa*) y asume que contamos con un conjunto de canciones de referencia para indexar.

1. Preparación e indexación de la base de datos de referencia

- **Carga de audio:** Leer los archivos de canciones de la base de datos. Con *librosa* se puede cargar fácilmente un archivo de audio a un arreglo numpy y su frecuencia de muestreo: `y, sr = librosa.load(path, sr=44100, mono=True)`. Es importante unificar a mono y una misma tasa (`sr`) para consistencia topical.com.
- **Cálculo del espectrograma:** Aplicar la STFT a cada canción. Con *librosa*: `S = librosa.stft(y, n_fft=2048, hop_length=512)` produce la matriz espectral compleja; usamos la magnitud: `S_mag = np.abs(S)`. Podemos convertir a escala logarítmica: `S_db = librosa.amplitude_to_db(S_mag)`.
- **Detección de picos:** Sobre `S_db`, encontrar picos locales. Esto puede hacerse de varias formas. Una simple: dividir las frecuencias en bandas (por ejemplo, dividir el

índice de frecuencia en 5 rangos) y para cada columna de tiempo tomar el índice de frecuencia de mayor valor en cada banda toptal.com. Alternativamente, usar un enfoque 2D: utilizar `scipy.signal.find_peaks` en cada columna para varios picos, o tratar cada pequeña región como imagen y detectar máximos. Incluso se puede utilizar `skimage.feature.peak_local_max` en el espectrograma 2D para obtener coordenadas (t,f) de máximos locales.

- **Generar hashes (fingerprints):** Con los picos identificados, generar las huellas. Si se siguen pares de picos al estilo Shazam, para cada pico “ancla” tomar otros picos dentro de una ventana de tiempo futura (ej. próximos 5 segundos) y formar tuplas `(f1, f2, Δt)` que luego se hashéan (por ejemplo usando una fórmula que combine estos números en uno solo, o aplicando un hash tipo SHA-1 a una cadena con f1,f2,dt) willdrevo.com. Si se sigue el método simplificado de *bandas*, se tomarían los N picos (uno por banda) de una ventana y se combinan en un hash entero toptal.com. Es importante cuantizar ligeramente los valores para robustez: por ejemplo, redondear frecuencias a 5-10 Hz y tiempos a, digamos, 20 ms antes de generar el hash toptal.com.
- **Almacenamiento en la base de datos:** Guardar cada hash junto con la identidad de la canción y el tiempo donde ocurrió. En un entorno de prototipo, se puede usar una estructura en memoria como un diccionario Python:
`hash_table[hash_value] = [(song_id, time), ...]`. Para algo más escalable, una base de datos relacional o NoSQL sería adecuada. Por ejemplo, una tabla SQL `fingerprints(hash VARBINARY(10), song_id INT, offset FLOAT)` con índice en `hash` willdrevo.com, o una tabla NoSQL (DynamoDB) con la hash como key de partición.

Al finalizar esta etapa, tendremos una colección de huellas para cada canción en la referencia. Conviene también guardar metadatos de las canciones (ID, nombre, artista) en otra estructura para traducir los resultados identificados.

2. Identificación de una canción desconocida (consulta)

- **Grabación/entrada de audio:** Desde la app móvil, se obtiene un fragmento de audio (por ejemplo 5 segundos capturados del micrófono). Este se envía al servidor o se guarda como archivo temporal.
- **Preprocesado similar:** El servidor Python recibe el audio y realiza los mismos pasos que con las canciones de la base: carga el audio (posiblemente ya recibido como PCM o stream), calcula su espectrograma con los mismos parámetros (mismo `n_fft`, `hop_length`, etc.), obtiene los picos dominantes y genera los hashes de la muestra desconocida.
- **Búsqueda de coincidencias:** Para cada hash calculado de la muestra, se consulta la estructura de huellas de la base de datos. Si usamos un diccionario en Python, sería simplemente `if hash in hash_table: candidates =`

`hash_table[hash]`. Esto devuelve una lista de pares (`song_id`, `time`) para cada ocurrencia de ese hash en canciones de la base topical.com. Se acumulan todas las coincidencias encontradas.

- **Agregación por canción y voto:** Con todas las coincidencias, se agrupan por `song_id` para contar cuántos hashes de la muestra aparecen en cada canción candidata. Además, se puede refinar calculando la diferencia entre los tiempos de la canción y los tiempos correspondientes en la muestra. En la canción correcta, todos esos offsets deberían ser casi iguales (indicando un alineamiento consistente) topical.com. Por ejemplo, si un hash de la muestra en $t=2.0s$ coincide con (`songX`, $t=50s$) $\Rightarrow$ offset $\sim 48s$; otro hash de la muestra en $t=3.0s$ coincide con (`songX`, $t=51s$) $\Rightarrow$ offset $\sim 48s$; muchos hashes dando $\sim 48s$ de offset sugieren que la muestra corresponde a la canción X empezando cerca del segundo 48.
- **Decisión:** Seleccionar la canción con mayor número de coincidencias y cuyo patrón de offsets sea coherente. Normalmente habrá un claro ganador. En caso de empate o duda, podría requerirse más audio de entrada para decidir, pero con ~ 5 segundos suele bastar si la base no tiene duplicados muy similares.
- **Salida:** Devolver el resultado (ID de canción encontrado) que la app luego traduce a nombre de canción/autor para mostrar al usuario.

Ejemplo con librerías: A modo de ilustración simplificada en código Python:

```
import librosa, numpy as np

# 1. Indexado de una canción de ejemplo
y, sr = librosa.load("songA.wav", sr=44100, mono=True)
S = np.abs(librosa.stft(y, n_fft=2048, hop_length=1024)) #
espectrograma magnitud
S_db = librosa.amplitude_to_db(S, ref=np.max)             # a
decibelios
peaks = [] # lista para (freq_idx, time_idx) de picos
# Detectar máximos locales por bloques (ejemplo: un pico por banda
en cada frame)
freq_bins, time_frames = S_db.shape
band_size = freq_bins // 5
for t in range(time_frames):
    for band in range(5):
        start = band * band_size
        end = (band+1) * band_size
        freq_slice = S_db[start:end, t]
        f_rel = np.argmax(freq_slice)
        f_idx = start + f_rel
```

```

        peaks.append((f_idx, t))
# Generar hashes a partir de pares de picos
fan_window = 15 # mirar 15 frames adelante (~15*hop_length/44100 s)
hashes = []
for i, (f1, t1) in enumerate(peaks):
    for j in range(i+1, len(peaks)):
        f2, t2 = peaks[j]
        if 0 < t2 - t1 <= fan_window:
            dt = t2 - t1
            # Crear hash combinando f1, f2, dt (aquí un simple
string hashable)
            h = f"{f1}|{f2}|{dt}"
            hashes.append((h, t1))
# Insertar en base de datos
song_id = "SongA"
for h, t in hashes:
    hash_table.setdefault(h, []).append((song_id, t))

```

El ejemplo anterior muestra una posible ruta: usando STFT de librosa, identificando picos por bandas (muy simplificado) y creando hashes de pares de picos. En producción se ajustarían muchos detalles (umbral de amplitud para considerar un pico, tamaño de ventana temporal para emparejar picos, un hash más compacto que una cadena, etc.). No obstante, capta la idea: **reducir el espectrograma a identificadores únicos almacenables**.

3. Optimización y consideraciones

Para un prototipo funcional con pocos cientos de canciones, este pipeline puede correr en una sola máquina. Sin embargo, a medida que crezca la base de datos, es crucial optimizar:

- **Estructura de datos para búsqueda:** En lugar de diccionario en Python (que es rápido pero mantenido en RAM), se puede usar una base de datos persistente. Una opción flexible es *SQLite* o *PostgreSQL* con índice en la columna hash; para mayor escala, *DynamoDB* (NoSQL en AWS) puede almacenar las entradas hash -> lista de canciones. *DynamoDB* permitiría realizar consultas de hash muy rápidas a escala web.
- **Reducción de huellas almacenadas:** Ajustar parámetros para evitar excesos. Por ejemplo, limitar el número de pares de picos generados por segundo de audio (fan-out) para controlar el total de hashes por canción. En la implementación de *Dejavu* (un proyecto open-source de fingerprinting) se generaban ~5000–10,000 hashes por canción de 3–4 min, lo cual es manejable. Aumentar demasiado el “fan-out” mejora recall pero aumenta la base de datos exponencialmente willdrevo.com. Se debe buscar un balance.

- **Precisión vs robustez:** El sistema clásico de Shazam funciona pre-IA, pero hay que mencionar que existen enfoques modernos con aprendizaje automático: por ejemplo, generar **vectores de embeddings** de audio (usando modelos pre-entrenados como VGGish o Wav2Vec) y usar búsquedas de similitud en espacios vectoriales (FAISS, etc.) superteams.ai. Aunque poderosos, estos métodos requieren más recursos y entrenamiento. Para un estudiante de física con conocimientos medios de programación, el enfoque basado en FFT y picos, como hemos descrito, es más transparente y suficiente para un prototipo exitoso (además de alinearse con la teoría de Fourier vista en clase).

Integración con una app Android y uso de la nube (AWS)

Una vez implementado el algoritmo en Python, el siguiente paso es **integrarlo con una aplicación móvil**. La arquitectura recomendada es cliente-servidor, apoyándose en servicios en la nube para el procesamiento pesado, dado que un smartphone tiene recursos limitados para calcular FFTs y manejar una gran base de datos localmente.

Flujo típico: La app Android graba audio desde el micrófono (usando las APIs de Android MediaRecorder, por ejemplo) durante unos segundos cuando el usuario solicita identificar una canción. Ese fragmento de audio se envía a un servicio en la nube que ejecuta el algoritmo de reconocimiento y devuelve el nombre de la canción.

En AWS (Amazon Web Services), esto se puede lograr de forma *serverless* sin tener que desplegar servidores dedicados, mediante los siguientes componentes:

- **Amazon S3:** Servicio de almacenamiento de archivos. La app podría subir el clip de audio (p. ej. en formato WAV/MP3) a un bucket S3, desde donde el backend lo procesará. Alternativamente, el audio puede enviarse directamente en la petición API (en cuyo caso S3 no es estrictamente necesario, pero es útil para almacenar la base de datos de huellas o archivos de referencia). Por ejemplo, podríamos almacenar en S3 todos los archivos de canciones de referencia o un archivo con las huellas precomputadas de cada canción.
- **AWS Lambda:** Funciones en la nube bajo demanda. Podemos implementar el **core** del algoritmo de identificación como una función Lambda escrita en Python. Esta función se activará cuando llegue una solicitud (por ej., vía API Gateway) o un nuevo objeto en S3 (el audio subido). La función Lambda cargaría el audio de la muestra, calcularía su fingerprint usando librerías (es posible incluir **librosa**, **numpy**, etc., quizás empaquetándolas como layer si exceden tamaño), consultaría la base de datos de huellas y determinaría la canción coincidente. Lambda tiene la ventaja de **escalar automáticamente** y de no requerir gestión de servidores; solo se paga por ejecución por milisegundos, lo cual para un uso infrecuente (p.ej., demostración) es muy económico.

- **AWS API Gateway:** Permite exponer una API HTTP/REST para que la app móvil invoque la función Lambda fácilmente. Por ejemplo, se puede configurar un endpoint POST `/identify` que reciba el audio (o un identificador del audio en S3) y dispare la Lambda de identificación. API Gateway actúa como puerta de enlace segura, manejando autenticación si hace falta, y simplifica la comunicación entre la app y la nube.
- **Base de datos de huellas en AWS:** Si la base de huellas es grande, podríamos almacenarla en un servicio gestionado. Dos opciones: una base NoSQL como **Amazon DynamoDB**, que es muy adecuada para buscar por clave (hash) de forma ultrarrápida y escalable, o una base SQL en **Amazon RDS** (Postgres/MySQL) si se prefiere SQL tradicional. DynamoDB sería conveniente por su modelo de clave-valor: podríamos tener la tabla `Fingerprints` con `hash` como clave de partición y la lista de canciones/timestamps como valor. Como las consultas serán siempre “dame entradas con esta clave hash”, DynamoDB rendiría muy bien. En un prototipo pequeño, incluso se podría cargar la estructura en memoria dentro de la Lambda (por ejemplo, desde un archivo JSON en S3) – aunque si crece a millones de huellas, esto ya no es viable por límites de memoria de Lambda.
- **AWS EC2 (opcional):** En caso de que la implementación con Lambda presente limitaciones (por ejemplo, tiempo de ejecución > 15 min en casos extremos, o dificultad para cargar librerías muy pesadas), se puede considerar desplegar un servicio en un servidor EC2 (una instancia virtual) corriendo un API Python (Flask, FastAPI) con el modelo de identificación precargado. No obstante, para el caso típico (identificación en segundos), Lambda suele ser suficiente. EC2 tendría sentido si quisiéramos, por ejemplo, un servicio en tiempo real muy afinado, multihilo en C++ para máximo rendimiento, pero para una demo educativa, **serverless evita mucho trabajo de infraestructura**.

Arquitectura recomendada: La app envía solicitud -> API Gateway -> Lambda -> (consulta base de datos) -> resultado a Lambda -> API Gateway -> app. Esto evita tener que construir desde cero un API personalizada; AWS proporciona los bloques ya hechos. No es necesario programar un servidor web completo, solo la lógica de la función. De hecho, existen ejemplos de reconocimiento de audio que usan exactamente esta combinación de Lambda + API Gateway para procesar audio en la nube de forma escalable.

Procesamiento en la nube vs en el dispositivo: Vale la pena destacar que delegar la comparación a la nube es beneficioso porque:

- La base de datos de huellas puede ser grande; mantenerla actualizada y hacer búsquedas locales en el teléfono sería inviable.
- El procesamiento FFT y extracción de picos puede consumir CPU y batería; es preferible hacerlo en un entorno robusto en la nube.

- Facilita actualizar la base de datos (agregar nuevas canciones) centralmente sin tener que actualizar cada dispositivo.

La latencia añadida por enviar ~5–10 segundos de audio y obtener respuesta de la nube suele ser aceptable (Shazam por ejemplo responde típicamente en 2–5 s). Con una arquitectura bien diseñada, nuestra demo podría identificar en pocos segundos también. Si se quisiese reducir la cantidad de datos enviados, una idea es calcular la huella en el móvil y solo enviar los hashes; sin embargo, eso implicaría implementar el algoritmo de fingerprint en Java/C++ en la app, lo cual es más complejo. Usar la nube nos permite aprovechar directamente nuestro código Python con librosa.

En cuanto a AWS específicos: un plan podría ser usar **Amazon API Gateway + Lambda + DynamoDB + S3** totalmente serverless. La Lambda podría montarse junto a un *AWS Lambda Layer* que contenga las dependencias de librosa/numpy para no superar el tamaño límite. DynamoDB manejaría la búsqueda de hashes de manera escalable. S3 almacenaría quizás los fingerprints precalculados o backups. Todo ello con un mínimo de mantenimiento y pago por uso. No haría falta construir APIs personalizadas en EC2 a menos que se requiera un control muy fino o persistencia continua en memoria de datos (aunque DynamoDB ya actúa como almacenamiento persistente altamente disponible).

Conclusiones

En resumen, el algoritmo base de Shazam se fundamenta en **fingerprinting acústico**: convertir la canción en una firma a partir de su contenido frecuencial más distintivo. La **base mínima** para que funcione incluye:

- Transformar el audio al dominio frecuencia-tiempo (espectrograma vía FFT),
- Extraer *picos* o componentes sobresalientes en ese espectrograma (frecuencias dominantes vs tiempo),
- Codificar esos patrones en huellas digitales (hashes compactos)
- Comparar eficientemente esas huellas con una base de datos de referencia usando estructuras indexadas de búsqueda.

Con este método, un breve fragmento de audio (unos segundos) puede identificarse buscando coincidencias de huellas en una gran colección de canciones toptal.com, gracias a la rapidez de las búsquedas por hash y al robusto esquema de coincidencia por constelaciones de picos. Para implementarlo, Python ofrece las herramientas científicas necesarias (NumPy, SciPy, librosa) para el análisis espectral y manejo de señales, mientras que una integración con Android se logra mejor mediante un servicio en la nube.

Se han señalado fuentes de **datasets públicos** como GTZAN y FMA que permiten construir una base de datos de prueba sin problemas de licencia, así como los componentes de AWS

idóneos para desplegar la solución de forma escalable y sin administrar servidores directamente (API Gateway, Lambda, DynamoDB, S3).

Este proyecto interdisciplinario combina fundamentos de física (análisis de Fourier), matemáticas (algoritmos de hashing, búsqueda) y ciencias de la computación (sistemas distribuidos, bases de datos) en una aplicación práctica. Con un enfoque claro y modular, es factible desarrollar un clon básico de Shazam que funcione como **prueba de concepto**: la aplicación móvil captura el sonido, y un backend Python en la nube realiza el reconocimiento devolviendo el nombre de la canción en cuestión. Este tipo de solución demuestra cómo principios de procesamiento de señales se traducen en tecnología cotidiana de reconocimiento musical, y brinda un excelente aprendizaje integrador de teoría y práctica.

Referencias utilizadas: Las explicaciones se basaron en la descripción pública del algoritmo de Shazam [toptal.com](https://toptal.com/toptal.com), en artículos técnicos sobre huellas acústicas [es.wikipedia.org](https://es.wikipedia.org/toptal.com), y en recursos de datasets musicales abiertos (GTZAN tensorflow.org, FMA [archive.ics.uci.edu](https://archive.ics.uci.edu/archive.ics.uci.edu)). Se han citado fragmentos clave para respaldar la información presentada. Estas fuentes, junto con la documentación de herramientas como librosa, pueden consultarse para profundizar en detalles de implementación. ¡Manos a la obra con la construcción del prototipo Shazam casero! 🎵